

课程笔记

CS146S Week 8: Automated UI and App Building

基于 Mihail Eric + Gaspar Garcia (Vercel AI Research) 授课内容整理

2025 年 11 月

课程: Stanford CS146S

学期: Fall 2025

讲次: 2 lectures (Mon + Fri)

网站: <https://themodernsoftware.dev>

目录

1	Web 应用开发的演进	2
1.1	传统 Web 技术栈	2
1.2	第一代低代码/无代码	2
1.3	AI 时代的应用构建	2
1.4	本章小结	2
2	App Builder 的架构原理	3
2.1	基本架构	3
2.2	v0 的引擎详解	3
2.2.1	五步流水线	3
2.3	处理模型的局限性	3
2.3.1	Stream Manipulation	3
2.3.2	训练与微调	3
2.4	本章小结	4
3	AI 应用构建的局限与展望	4
3.1	当前的局限	4
3.2	实际影响	4
3.3	本章小结	4
4	总结与延伸	4
4.1	关键点	4
4.2	拓展阅读	5

1 Web 应用的演进

1.1 传统 Web 技术栈

现代 SaaS 开发以 Web 应用为中心已有超过二十年。传统的技术栈经历了多代演进：

- **LAMP**: Linux + Apache + MySQL + PHP
- **MERN**: MongoDB + Express + React + Node.js
- **JAM**: JavaScript + APIs + Markup
- **Serverless**: 无服务器架构

无论哪种技术栈，架构都遵循经典的三层模式：客户端 (client-side) → 服务端 (server-side) → 数据库 (database)。

1.2 第一代低代码/无代码

- Wix、Squarespace、Webflow 等可视化建站工具
- 降低了建站门槛，但定制能力有限

1.3 AI 时代的应用构建

从写代码到说需求

AI 应用构建工具的核心转变：从“写代码构建应用”到“用自然语言描述需求，AI 生成应用”。这比第一代 no-code 平台更加用户友好，同时保留了更强的定制能力。

新一代 AI 应用构建工具：

- **Lovable**: AI 驱动的全栈应用生成
- **Replit**: 在线 IDE + AI 编程
- **Vercel v0**: 自然语言 → React UI 组件
- **Base44**: 低代码 + AI
- **Cursor / Claude Code**: 通用 AI 编码工具也可用于应用构建

工程师角色的转变

AI 应用构建工具正在使工程师变得更加**跨职能** (cross-functional)：开发者现在也承担了设计 (design) 和产品管理 (product management) 的职责。同时，非技术人员也可以构建应用——“Anyone can build an app now.”

1.4 本章小结

Web 应用开发从 LAMP 到 Serverless，再到 AI 应用构建，每一代都降低了门槛、提高了效率。AI 时代的核心转变是：自然语言成为“编程语言”。

2 App Builder 的架构原理

2.1 基本架构

App builder 的核心架构通常基于 WebContainer 技术——在浏览器内运行一个完整的开发环境，支持代码编辑、编译和预览。

2.2 v0 的引擎详解

Vercel v0 的 Head of AI Research Gaspar Garcia 详细介绍了 v0 的内部架构：

2.2.1 五步流水线

1. **Intent Understanding**: 自然语言处理提取结构化需求——组件、布局、交互、数据流
2. **Context Assembly**: 收集相关信息——设计系统 token、现有组件、API schema、无障碍要求
3. **Code Generation**: LLM 生成生产质量的 React 组件 (TypeScript 类型、响应式设计、最佳实践)
4. **Validation & Testing**: 自动化检查无障碍合规、响应式行为和代码质量
5. **Human-in-the-Loop**: 工程师审查、优化，反馈用于训练模型

Agent 是 Workflow

Gaspar 强调了一个核心设计理念：Agent 本质上是 workflow（工作流）。v0 的“智能”不是来自单一的大模型调用，而是来自精心设计的多步骤流水线。每个步骤都有明确的输入、输出和验证标准。

2.3 处理模型的局限性

2.3.1 Stream Manipulation

v0 创建了一个子系统，可以在 LLM 的输出流到达用户之前进行拦截和修改。这解决了“有时候仅调整 prompt 不够”的问题——直接在输出层面进行纠正。

2.3.2 训练与微调

前沿模型实验室不会专门为 Next.js 16 Web 应用创建定制的代码生成管线。v0 通过自有的训练和微调 (fine-tuning) 管线，构建了 composite model family，确保生成的代码符合最新的框架规范。

Composite Model Family

v0 不依赖单一的通用模型，而是使用一组经过针对性微调的模型组合 (composite model family)。不同的模型负责不同的子任务，组合起来实现超越单一模型的整体效果。详见 Vercel 博客：<https://vercel.com/blog/v0-composite-model-family>

2.4 本章小结

现代 App Builder 的核心是精心设计的多步骤 workflow，而非单一的 LLM 调用。Stream manipulation 和 composite model family 是解决通用模型局限性的关键工程手段。

3 AI 应用构建的局限与展望

3.1 当前的局限

“Everything is awesome until it breaks”

AI 应用构建的最大挑战：当一切正常时体验很好，但一旦出错就回到了原点——开发者仍然需要理解底层代码才能调试和修复。

- **Prompt 只是建议**：不是每个用户都理解这一点
- **安全性**：历史上出现过安全问题
- **同质化**：如何防止所有 AI 生成的应用看起来一模一样？
- **复杂度上限**：这些应用到底能做到多复杂？

3.2 实际影响

AI 应用构建工具正在三个层面产生影响：

- **Startup Velocity**：初创公司在数天而非数月内发布 MVP
- **Enterprise Modernization**：大型组织在投入工程资源前先用 AI 原型验证
- **Agency Efficiency**：设计机构在启动会议后数小时内交付交互式原型

3.3 本章小结

AI 应用构建工具显著降低了从想法到原型的时间，但“一旦出错回到原点”的问题提醒我们：理解底层技术仍然不可或缺。

4 总结与延伸

Week 8 展示了 AI 如何变革应用构建：从传统 LAMP 栈到 AI 驱动的自然语言 → 应用的范式。v0 的架构详解揭示了 App Builder 的“魔法”背后是精心设计的 workflow 和 composite model。

4.1 关键点

- Web 开发从 LAMP → MERN → JAM → Serverless → AI App Builder 演进
- AI 使工程师更加跨职能，非技术人员也可以构建应用
- v0 架构：Intent → Context → Code Gen → Validation → Human Review
- “Agent 是 Workflow”——核心不是单一模型调用，而是多步流水线

- Stream manipulation 和 composite model family 解决通用模型的局限
- 最大局限：出错时仍需人类理解底层代码来调试

4.2 拓展阅读

- Vercel v0: <https://v0.dev> (学生免费: <https://v0.app/students>)
- Vercel AI SDK: <https://ai-sdk.dev/docs/agents/overview>
- Composite Model Family 博文: <https://vercel.com/blog/v0-composite-model-family>
- Lovable: <https://lovable.dev>
- Bolt.new (StackBlitz): <https://bolt.new>
- Replit: <https://replit.com>